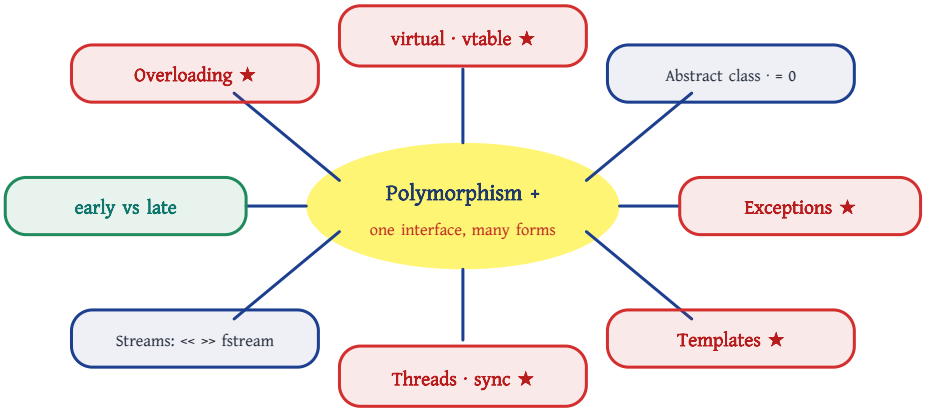
Polymorphism, Exceptions, Templates & Threads

Virtual · Exceptions · Templates · I/O · Threads: exam-ready in 11 sheets



↑ the whole lesson on one page: red = high-yield, green = the idea everything else hangs on

Syllabus coverage

The OOP syllabus continues with "**Polymorphism: overloading, abstract classes, virtual functions and overriding; Exceptions; Object Oriented I/O; Template functions and classes; Multi-threaded Programming.**" Classes, access and static members are in the first lesson; constructors and inheritance in the second.

High-yield list

① Overload: compile time; override: run time
② Return type alone ≠ overload
③ No overload: `:: . .* ?: sizeof`
④ Java methods virtual by default
⑤ ≥ 1 pure virtual ⇒ abstract
⑥ Polymorphic base: virtual dtor
⑦ `finally` runs either way
⑧ Checked = compiler-enforced
⑨ Templates: compile-time copies
⑩ `start()` = new thread; `run()` no

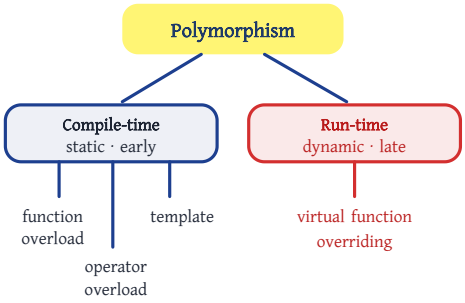
Legend: **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Polymorphism & overloading … p. 2
✓ Virtual functions & overriding … p. 3
✓ Abstract classes, virtual destructor … p. 4
✓ Exceptions: C++ and Java … p. 5
✓ Templates and generics … p. 6
✓ Object-oriented I/O: streams … p. 7
✓ Threads: creation & life cycle … p. 8
✓ Synchronization & deadlock … p. 9
✓ Night-before sheet … p. 10
✓ Practice MCQs + written … p. 11

1 Polymorphism: Compile-time vs Run-time; Overloading

Definition: **Polymorphism** ("one interface, many forms") lets the same name do different work depending on the types involved. **Binding** = connecting a call to a function body. **Early (static) binding** happens at compile time — overloading and templates. **Late (dynamic) binding** happens at run time — overriding through a base pointer or reference.



two families, four tools

Function overloading rules: Same name, same scope; parameter lists differ in **number**, **type** or **order**. **Return type alone** is not enough: `int f(int)` + `double f(int)` = compile error. The compiler ranks: exact match → promotion (`char→int`, `float→double`) → standard conversion → user conversion; two equally good matches = **ambiguity error**. Java also lets a subclass overload an inherited method.

Point	Compile-time polymorphism	Run-time polymorphism
Also called	static, early binding	dynamic, late binding
Decided by	argument list / types	actual object type
Mechanism	overloading, templates	`virtual` + overriding
Needs inheritance	no	**yes**

```
class Complex {
  double re, im;
public:
  Complex(double r=0, double i=0)
      : re(r), im(i) {}
  Complex operator+(const Complex& c) const {
    return Complex(re + c.re, im + c.im);
  }
};
Complex a(1,2), b(3,4);
Complex c = a + b;  // a.operator+(b) → 4+6i
```

Worked example ✍ which f?: `void f(int); void f(double);`
`f('a')` → promotion → **f(int)**
`f(2.5f)` → float promotes → f(double)
`f(5L)` → long converts to both → **ambiguous**

C++ operators	Operators
Cannot be overloaded	`::` · `.` · `.*` · `?:` · `sizeof` · `typeid` · `alignof` · the `_casts` · `#` `##`
Only as a **member**	`=` · `()` · `[]` · `->` (and conversion operators)
Usually a **friend**	`<<` `>>` for streams; symmetric `a + 2` / `2 + a`

Memory trick: The five you can't touch: "**Scope, Dot, Dot-star, Question, Size**" — `::` `.` `.*` `?:` `sizeof`.

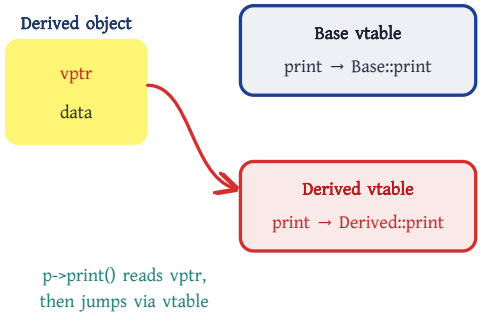
Exam trap: **Java has no user operator overloading**; `+` on String is built in. `f(int, int = 0)` + `f(int)`: `f(5)` is ambiguous.

Rules: Precedence, associativity and arity never change; no new operators (`**`); one operand must be a user type.

2 Virtual Functions, vtable and Overriding

Definition: **Overriding** = a derived class redefines an inherited method with the **same signature**. A C++ **virtual function** is resolved by the **object's** type, not the pointer's, so `Base* p = new Derived; p->f();` runs `Derived::f`. Without `virtual` the pointer's type decides (static binding). The compiler implements this with a **vtable** per class and a hidden **vptr** per object.

```
class Base { public:
  void show() { cout << "Base "; }
  virtual void print() { cout << "BasePrint"; }
};
class Derived : public Base { public:
  void show() { cout << "Derived "; }
  void print() override
               { cout << "DerivedPrint"; }
};
Base* p = new Derived;
p->show();   // Base         (static)
p->print();  // DerivedPrint (dynamic)
// output: Base DerivedPrint
```



one vtable per class, one vptr per object

Point	Overloading	Overriding
Where	same class	base and derived class
Signature	must **differ**	must be the **same**
Return type	anything	same or covariant (a subclass type)
Binding	compile time	run time
Access (Java)	any	same or wider, never narrower
static / private / final	can be overloaded	**not overridable**; static is *hidden*

Language	Virtual by default?	Keywords	Block overriding
C++	no	`virtual`, `override`	`final` (C++11)
Java	**yes** (non-static, non-private)	`@Override` (optional)	`final`
C#	no	`virtual` / `override`	`sealed`

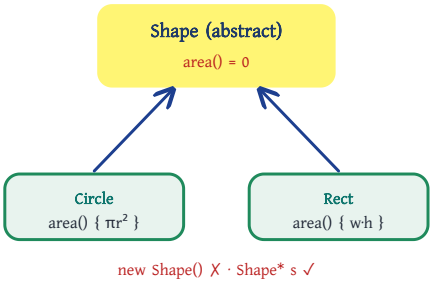
Memory trick: "The pointer picks the menu; with `virtual`, the object picks the dish."

Exam trap: **Constructors cannot be virtual**; a virtual call inside one runs the *base* version. Java overriding may not **reduce access** or throw broader checked exceptions.

Worked example ✍ sizeof: 64-bit: `class A { int x; };` → 4. Add one virtual function → **16** (8-byte vptr + 4 + padding). More virtual functions: still one vptr.

3 Abstract Classes and Virtual Destructors

Definition: A **pure virtual function** is declared with `= 0` — `virtual double area() = 0;` — and says "every subclass must supply this". A class with **at least one** pure virtual function is an **abstract class**: no objects of it can be created, but pointers and references to it can. A derived class that does not override every pure virtual function is abstract too. Java and C# use the keyword `abstract`.



dashed = cannot be instantiated

```
class Shape { public:
  virtual double area() = 0;  // pure
  virtual ~Shape() {}         // virtual dtor
};
class Circle : public Shape { double r;
public:
  Circle(double r) : r(r) {}
  double area() override { return 3.14159*r*r; }
};
Shape* s[2] = { new Circle(1), new Rect(2, 3) };
double total = 0;
for (auto x : s) total += x->area();
// Shape t;  → error: abstract class
```

Can an abstract class …	C++	Java
have constructors, fields, normal methods?	yes (derived class runs the constructor)	yes
have **zero** abstract methods?	no — abstract only if ≥ 1 pure virtual	**yes**, if marked `abstract`
give the abstract method a body?	yes, defined outside: `Shape::area()`	no

Same in Java: `abstract class Shape { abstract double area(); }` · `class Circle extends Shape {…}` · `Shape s = new Circle(1);` is legal — the reference type may be abstract.

Virtual destructor: `Base* p = new Derived; delete p;` — if `~Base()` is **not** virtual, only the base destructor runs (formally undefined behaviour) and the derived part leaks. Rule: **a class with any virtual function needs a virtual destructor**. Order with it: `~Derived()` then `~Base()`.

Object slicing: `Derived d; Base b = d;` copies **only the Base part**; the derived fields are "sliced off" and `b.print()` calls `Base::print`. Polymorphism works only through **pointers or references**, never by value. Java has no slicing: variables hold references.

Memory trick: "**`= 0` means zero objects.**" Put `= 0` anywhere in a class and it can no longer be built.

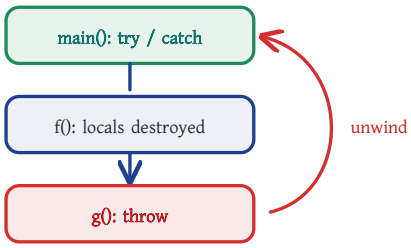
Exam trap: "Abstract class contains only abstract methods" is **false**. C++ has no `abstract` keyword. Abstract class vs interface: first lesson.

Worked example ✍ total area: Circle(1) + Rect(2,3) = 3.14159 + 6 = **9.14**, each `area()` chosen at run time.

4 Exception Handling: C++ and Java

Definition: An **exception** is an abnormal event at run time (divide by zero, bad index, missing file) that breaks the normal flow. `try` guards a block, `throw` raises an exception object, `catch` handles a matching type. A `throw` triggers **stack unwinding**: frames are popped and their locals destroyed until a handler matches; none → `std::terminate()` (C++) or the thread dies with a stack trace (Java).

```
double divide(int a, int b) {
  if (b == 0) throw runtime_error("b = 0");
  return (double)a / b;
}
try {
  cout << divide(10, 0);   // skipped
} catch (const runtime_error& e) {
  cout << e.what();        // b = 0
} catch (...) {             // any type
  cout << "unknown";
}
```



stack unwinding

Keyword	C++	Java
`throw`	any type; `throw;` rethrows	`throw new X()` — a Throwable
`throws`	none; `noexcept` = will not throw	header lists checked exceptions
`finally`	**none** — destructors clean up (RAII)	runs **whether or not** one occurs
catch-all	`catch (...)`, must be last	`catch (Exception e)`; multi-catch `A | B`

Java class	Kind	Examples
Throwable → **Error**	unchecked	OutOfMemoryError, StackOverflowError (don't catch)
Exception → **RuntimeException**	**unchecked**	Arithmetic, NullPointer, ArrayIndexOutOfBounds, ClassCast
other **Exception**s	**checked**	IOException, FileNotFound, SQLException, InterruptedException

Checked = the compiler forces you to catch it or declare it with `throws`. C++: `std::exception` → *logic_error* (out_of_range, invalid_argument), *runtime_error* (overflow_error, underflow_error), *bad_alloc*, *bad_cast*; all have `what()`.

Memory trick: "**Catch the child before the parent.**" The first matching handler wins, so derived types go first. "**Checked = Compiler-checked.**"

Exam trap: `catch (Exception e)` above a subclass catch = **compile error** in Java. `finally` is skipped only by `System.exit()` or a crash; a `return` in finally overrides the try's.

Worked example ✍ trace: `int[] a = new int[3]; try { a[3]=10; print("X"); } catch {print("Y");} finally {print("Z");} print("W");` → **YZW**

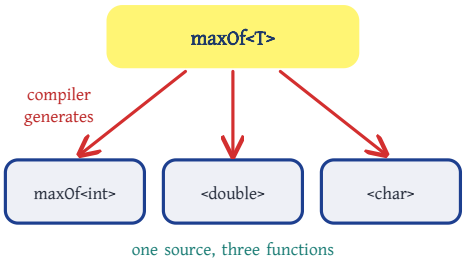
5 Template Functions and Classes; Java Generics

Definition: A **template** is a blueprint written once with a type parameter `T`. When code uses it with a concrete type, the compiler **instantiates** it — generates a separate function or class for that type — at **compile time**. This is **parametric (generic) polymorphism**; the whole STL (`vector<T>`, `sort`, `map<K,V>`) is built on it.

```
template <typename T>
T maxOf(T a, T b) { return a > b ? a : b; }

cout << maxOf(3, 7)        // T = int
     << " " << maxOf(2.5, 1.5)
     << " " << maxOf('a', 'z');
// output: 7 2.5 z
// maxOf(3, 2.5)  → error: T int or double?
// maxOf<double>(3, 2.5) → 3
```

```
template <class T, int N = 100>
class Stack {
  T data[N];  int top = -1;
public:
  void push(const T& x) {
    if (top == N - 1) throw overflow_error("full");
    data[++top] = x;
  }
  T pop() {
    if (top < 0) throw underflow_error("empty");
    return data[top--];
  }
};
Stack<int> si;  Stack<string, 10> ss;
```



instantiation at compile time

Specialization: A **full specialization** replaces the template for one exact type: `template<> const char* maxOf(const char* a, const char* b)` using `strcmp` — without it two C strings are compared by **address**. Class templates may also be **partially** specialized (`Stack<T*>`); function templates cannot. Template code lives in the **header**: the compiler needs the full body to instantiate it.

Point	C++ template	Java generics
Resolved	compile time, per type	compile time, then **type erasure**
Code copies	one per type (code bloat)	one class (`Object` inside)
Primitive args	yes: `Stack<int>`	no: `List<Integer>`, never `<int>`
Extras	non-type params `<int N>`, specialization	bounds `<T extends Comparable<T>>`, wildcard `?`

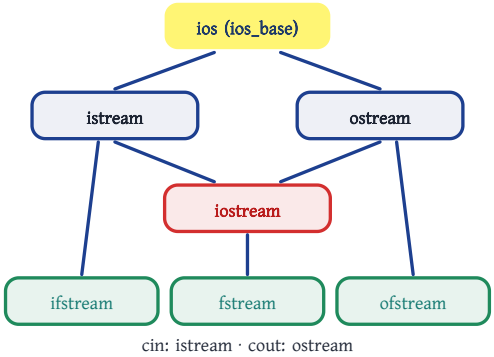
Memory trick: "**Templates are cookie cutters**": one cutter, a new cookie for every dough (type). Java generics: one cookie with a label.

Exam trap: Java generics are **not** templates: after erasure `List<String>` and `List<Integer>` share **one class**, so `new T()` is illegal.

Worked example ✍ Java: `static <T> void show(T v)`: `show(100)` autoboxes to Integer. `Box<String> b = new Box<>();` — the diamond infers String.

6 Object-Oriented I/O: Streams

Definition: In OO I/O, data moves through **stream objects**: a stream is a sequence of bytes flowing from a source (keyboard, file) or to a sink (screen, file). C++ supplies a class hierarchy in `<iostream>`, `<fstream>`, `<sstream>`; `<<` (**insertion**) and `>>` (**extraction**) are overloaded operators, so user classes can join in.



iostream: multiple inheritance

Object	Class	Device	Buffered
`cin`	istream	keyboard	yes
`cout`	ostream	screen	yes
`cerr`	ostream	error	**no**
`clog`	ostream	error log	yes

Mode	Meaning
`ios::in · out`	read · write
`ios::app`	every write at the end
`ios::ate`	open, seek to end once
`ios::trunc`	empty the file first
`ios::binary`	no text translation

```
class Point { int x, y;
 public:
  Point(int a=0, int b=0) : x(a), y(b) {}
  friend ostream& operator<<(ostream& os,
                             const Point& p) {
    return os << "(" << p.x << ","
              << p.y << ")";
  }
};
ofstream out("pts.txt", ios::app);
out << Point(2, 3) << endl;  // (2,3)
out.close();
ifstream in("pts.txt");  string line;
while (getline(in, line)) cout << line;
```

Manipulator	Effect
`endl`	newline + **flush**
`setw(n)`	width, **next** item only
`setprecision(n)`	digits (after point with `fixed`)
`setfill(c)`	padding character
`hex oct dec`	number base (sticky)

ofstream opens with `out | trunc` by default; check a file with `is_open()`, `eof()`, `fail()`.

Java I/O: **Byte** streams: InputStream / OutputStream (FileInputStream). **Character** streams: Reader / Writer (FileReader, FileWriter, BufferedReader with `readLine()`). `System.in` is an InputStream, `System.out` a PrintStream; `Scanner` parses tokens (`nextInt()`, `nextLine()`). Close files with **try-with-resources**; save whole objects with ObjectOutputStream (class must be Serializable).

Memory trick: "Arrows point where the data goes": `cout << x` pushes x out; `cin >> x` pulls into x.

Exam trap: `operator<<` must be a **non-member** (friend) returning `ostream&` for chaining. `cin >> s` stops at a space.

Worked example ✍️: `cout << setw(5) << 42 << 7;` → " 42 7": setw pads only 42.

7 Threads: Creation and Life Cycle

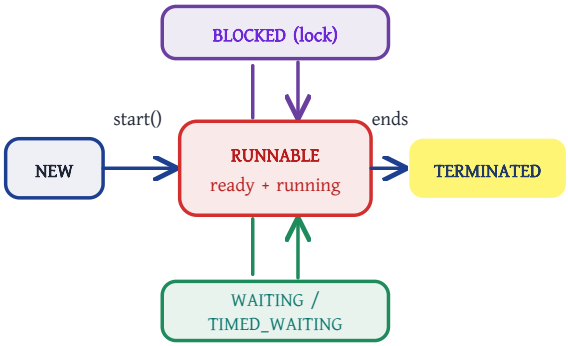
Definition: A **thread** is the smallest unit of CPU scheduling — a "lightweight process". **Multi-threading** runs two or more threads inside one process; they share its code, data and heap, but each has its own **stack, registers and program counter**. One core: threads run **concurrently** (interleaved); many cores: in **parallel**. Uses: responsive GUIs, servers, background downloads.

Point	Process	Thread
Memory	own address space	shares the process's memory
Create / switch	heavy, slow	**light, fast**
Communication	IPC: pipes, sockets	shared variables (need locks)
One crashes	others unaffected	can kill the whole process

```
// Way 1: extend Thread
class MyThread extends Thread {
  public void run() { System.out.println("T1"); }
}
// Way 2: implement Runnable (preferred)
class Task implements Runnable {
  public void run() { System.out.println("T2"); }
}
new MyThread().start();
new Thread(new Task()).start();
new Thread(() -> System.out.println("T3"))
    .start();              // lambda (Java 8)
```

start() vs run(): `start()` asks the JVM for a **new** thread, which then calls `run()`. Calling `run()` directly is an ordinary call on the **current** thread. `start()` twice → IllegalThreadStateException. **Runnable is preferred**: the class can still extend another class (single inheritance), and one task can be shared by many threads. The order in which threads print is **not guaranteed**.

Method	Does
`sleep(ms)`	pause; **keeps** locks
`join()`	wait for it to end
`yield()`	hint: let others run
`wait()`	**releases** the lock
`setPriority`	1 … **5** … 10
`setDaemon`	background thread



Java Thread.State: six states

Memory trick: "**New Runners Block, Wait, Time-wait, Terminate**" — the six Java states in enum order.

Exam trap: Java has **no RUNNING state**: RUNNABLE = ready + running. Waiting for a lock = BLOCKED; `wait()/join()` = WAITING; `sleep(ms)` = TIMED_WAITING.

Worked example ✍️: `t.run(); print("M");` → run's output always first (same thread). `t.start(); print("M");` → order **unpredictable**.

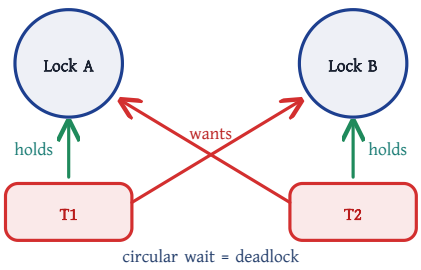
8 Synchronization, Race Conditions and Deadlock

Definition: A **race condition** occurs when two threads access shared data, at least one writes, and the result depends on the timing. The code that touches shared data is the **critical section**; **synchronization** guarantees **mutual exclusion** — only one thread in it at a time. `count++` looks like one step but is three (read, add, write), so two threads can lose an update.

```java
// Java: monitor lock on 'this'
class Counter {
  private int count = 0;
  public synchronized void increment() {
    count++;
  }
  public int get() { return count; }
}
// or: synchronized (lock) { count++; }
// or: AtomicInteger c; c.incrementAndGet();
```

```cpp
// C++11: <thread>, <mutex>
int count = 0;  mutex m;
void work() {
  for (int i = 0; i < 1000; i++) {
    lock_guard<mutex> g(m);  // RAII lock
    count++;
  }
}
thread t1(work), t2(work);
t1.join();  t2.join();   // wait for both
cout << count;           // always 2000
```

Tool	What it gives
Mutex (lock)	one owner at a time; the locker must unlock
Semaphore	counter: up to *n* threads; binary semaphore ≈ mutex (but no ownership)
Monitor	lock + waiting in one object: `synchronized` + `wait/notify`
`volatile` (Java)	visibility of the latest value; **not atomicity** — `count++` still races
`join` / `detach` (C++)	wait for it / let it run alone; neither → `std::terminate`



each holds one, waits for the other

Deadlock: 4 Coffman conditions: ① **Mutual exclusion** — a resource is held by one thread. ② **Hold and wait** — hold one, wait for another. ③ **No preemption** — a lock cannot be taken away. ④ **Circular wait** — T1 → T2 → … → T1. All four are needed; break one and deadlock is impossible — usually a **fixed lock order** (always A before B). **Starvation** = never gets a turn; **livelock** = busy but no progress.

Memory trick: Coffman = "**My Hat, No Coat**": Mutual exclusion, Hold & wait, No preemption, Circular wait.

Exam trap: `wait()/notify()` are **Object** methods, called only while holding the lock (else IllegalMonitorStateException).

Worked example ✍ lost update: count = 5. T1 reads 5, T2 reads 5, T1 writes 6, T2 writes 6 → final **6**, not 7. Without the lock, two × 1000 increments give ≤ 2000.

Night-Before-the-Exam Sheet

Fact sheet:

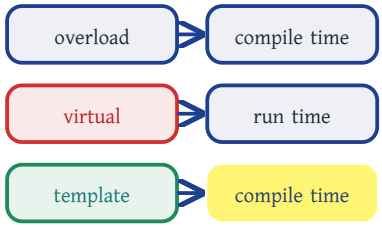
Overloading → compile time, early binding	Overriding + virtual → run time, late binding
Return type alone cannot overload	No overload: `:: . .* ?: sizeof`
Member only: `= () [] ->`	Java: no user operator overloading
vtable per class, vptr per object	Constructor never virtual; destructor should be
`virtual f() = 0` → abstract class	Polymorphism via pointer/reference; by value = slicing
C++ has no finally (RAII); Java finally always (except System.exit)	Checked: IOException, SQLException
Unchecked: RuntimeException + Error	Templates: instantiated per type at compile time
Java generics: type erasure, no primitives	`endl` = newline + flush; `cerr` unbuffered
Thread shares heap; own stack + PC	Java: 6 thread states; priority 1–10, default 5

Top 12 one-liners for MCQs

1. Polymorphism = one interface, many forms
2. Function overloading = static polymorphism
3. Java methods are virtual by default
4. C# needs `virtual` + `override`; `sealed` stops it
5. Catch derived exceptions before base
6. `catch (...)` catches anything, goes last
7. `throw` raises; `throws` declares
8. `noexcept` violated → `std::terminate`
9. STL is built on templates
10. `<<` overload = friend, returns `ostream&`
11. `start()` new thread; `run()` same thread
12. Deadlock needs all four Coffman conditions

Top traps: return type ≠ overload · virtual call in a constructor runs the base version · abstract class may have constructors · Java has no RUNNING state · `volatile` ≠ atomic · `sleep()` keeps the lock, `wait()` releases it · catch(Exception) before a subclass = compile error.

All tricks together: Scope, Dot, Dot-star, Question, Size · pointer picks the menu, object picks the dish · `= 0` means zero objects · catch the child before the parent · cookie cutters · arrows point where data goes · New Runners Block… · My Hat, No Coat.



Worked example ✍ one keyword changes the output: In the Base/Derived program, remove `virtual` from `print()`: `p->print()` is now bound to the pointer type, so the output becomes "**Base BasePrint**" instead of "Base DerivedPrint".

✓ Practice MCQs (exam pattern, 10 marks)

1. Function overloading is resolved at:
(a) compile time (b) run time
(c) link time (d) load time

2. Which operator cannot be overloaded in C++?
(a) `+` (b) `[]`
(c) `==` (d) `::`

3. A C++ class with at least one pure virtual function is:
(a) a friend class (b) a template class
(c) an abstract class (d) a static class

4. In Java, which block runs whether or not an exception occurs?
(a) catch (b) finally
(c) throw (d) throws

5. To begin a new thread in Java, call:
(a) `run()` (b) `init()`
(c) `start()` (d) `execute()`

6. Which is an unchecked exception in Java?
(a) IOException (b) SQLException
(c) EOFException (d) ArithmeticException

7. Why make a base-class destructor virtual?
(a) so `delete` via `Base*` runs `~Derived` (b) to make the class abstract
(c) to block copying (d) to speed up `delete`

8. C++ templates are instantiated at:
(a) run time (b) compile time
(c) load time (d) link time only

9. How many states does Java's `Thread.State` define?
(a) 6 (b) 4
(c) 5 (d) 7

10. Which is **not** a necessary condition for deadlock?
(a) mutual exclusion (b) hold and wait
(c) circular wait (d) preemption

Answer key → 1-a · 2-d · 3-c · 4-b · 5-c · 6-d · 7-a · 8-b · 9-a · 10-d

Written question: Distinguish compile-time and run-time polymorphism. Explain how a virtual function gives run-time polymorphism in C++, with an example and the vtable mechanism. Why should the destructor of a polymorphic base class be virtual? 10 marks

1. **Distinction (3)** — compile-time: overloading / templates, chosen from the argument list, early binding, fast; run-time: overriding of virtual functions through a base pointer or reference, chosen by the object's type, late binding, needs inheritance. 3
2. **Example (3)** — `Base` with `virtual void print()`, `Derived` overriding it; `Base* p = new Derived; p->print();` prints the derived version; without `virtual`, the base version. 3

3. **Mechanism (2)** — each class with virtual functions has a vtable of function addresses; each object has a hidden vptr to its class's vtable; the call reads vptr, indexes the table and jumps. 2
4. **Virtual destructor (2)** — `delete p` with a non-virtual `~Base` runs only the base destructor (undefined behaviour, leak); a virtual one runs `~Derived` then `~Base`. 2

Before the exam ✓

- ✓ Can list the non-overloadable operators
- ✓ Can trace a virtual / non-virtual call
- ✓ Can write an abstract class in C++ and Java
- ✓ Can order catch blocks and trace finally
- ✓ Can write a function and class template
- ✓ Can overload << for a class
- ✓ Can draw the six Java thread states
- ✓ Can state the four deadlock conditions